DEVIXO SOLUTIONS

Data Science Internship Program

Task 02 – Data Cleaning & Feature Engineering

Sample Superstore Sales Data

Student Name	Dua Sajid
Internship Domain	Data Science
Task Number	Task 02
Dataset	Sample Superstore (Kaggle) – 9,994 records (rows), 21 columns
GitHub repository link	https://github.com/duasajid65-cloud/DevixoSolutions-Internship

Objective

Turn the raw Superstore sales data into a clean, analysis-ready dataset and add new features that make it more useful. The work covers missing values, duplicates, outliers, data types, encoding, scaling, new features, before-and-after statistics and a reusable Scikit-learn preprocessing pipeline.

Problem Statement

Raw data is rarely ready for analysis or modelling. Dates are stored as text, a few columns hold extreme values, categories are words instead of numbers and the numeric columns sit on very different scales.

These issues have to be fixed, and every fix explained, before any analysis or prediction model is built.

Dataset Description and Source

Source: Sample Superstore retail sales data (Kaggle)

Size: 9,994 records (rows) and 21 columns, well above the 2,000-record minimum.

Column group	Columns
Order details	Row ID, Order ID, Order Date, Ship Date, Ship Mode
Customer and location	Customer ID, Customer Name, Segment, Country, City, State, Postal Code, Region
Product	Product ID, Category, Sub-Category, Product Name
Measures	Sales, Quantity, Discount, Profit

Technologies Used

Python, Pandas, NumPy, Matplotlib, Scikit-learn and Jupyter Notebook.

Implementation Steps

1. Load the dataset and inspect its structure (Part 1).
2. Generate descriptive statistics before preprocessing (Part 2).
3. Identify and handle missing values (Part 3).
4. Check for and remove duplicate records (Part 4).
5. Detect and handle outliers (Part 5).
6. Correct incorrect data types (Part 6).
7. Create eight new features (Part 7).
8. Encode categorical variables (Part 8).
9. Standardize the numerical features (Part 9).
10. Generate descriptive statistics after preprocessing and visualize them (Part 10).
11. Build a reusable Scikit-learn preprocessing pipeline and save the cleaned dataset (Part 11).

Data Import and Inspection (Part 1)

The data was loaded with pd.read_csv() using latin-1 encoding, and an untouched copy was kept for the before-and-after comparison. It has 9,994 rows and 21 columns, and all 21 columns have 9,994 non-null values.

Row ID, Postal Code and Quantity are integers, Sales, Discount and Profit are floats, and the other 15 columns, including both dates, are text. The dates and Postal Code are corrected in Part 6.

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
```

```python
df = pd.read_csv("superstore.csv", encoding = 'latin-1')
df_raw = df.copy()          # keep an untouched copy for the before/after comparison
```

Descriptive Statistics Before Preprocessing (Part 2)

Column	Mean	Std	Min	Max
Sales	229.86	623.25	0.44	22,638.48
Quantity	3.79	2.23	1	14
Discount	0.16	0.21	0.00	0.80
Profit	28.66	234.26	−6,599.98	8,399.98

Statistics of the raw numeric columns.

Why: these numbers are the “before” picture, to be compared with the “after” statistics in Part 10. Sales has a mean of 229.86 but a median of only 54.49, and its maximum is about 108 times the 75th percentile (209.94). Profit shows the same gap between its 75th percentile (29.36) and its maximum (8,399.98). Both point to outliers.

Missing Values (Part 3)

```python
# -----Check missing values-----
print(df.isnull().sum())


# -----Fill missing numerical values with the median-----
for col in df.select_dtypes(include = "number").columns:
    df[col] = df[col].fillna(df[col].median())


# -----Fill missing text values with the most common value-----
for col in df.select_dtypes(include = "object").columns:
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].mode()[0])

print("\n---Missing values handled.---")
```

```
Row ID          0
Order ID        0
Order Date      0
Ship Date       0
Ship Mode       0
Customer ID     0
Customer Name   0
Segment         0
Country         0
City            0
State           0
Postal Code     0
Region          0
Product ID      0
Category        0
Sub-Category    0
Product Name    0
Sales           0
Quantity        0
Discount        0
Profit          0
dtype: int64

---Missing values handled.---
```

Checking for missing values and filling any gaps

Result: no missing values were found in any column, so no rows were changed. The median fill (numbers) and most-common-value fill (text) are kept so the code also works on new data with gaps.

Duplicate Records (Part 4)

```
dup_df = df.drop(columns = "Row ID").duplicated()
print("Duplicates Before:", dup_df.sum())
df = df[~dup_df]
print("Shape after removing duplicates:", df.shape)
```

✓ 0.0s

```
Duplicates Before: 1
Shape after removing duplicates: (9993, 21)
```

Checking for and removing duplicate rows

Result: 1 duplicate was found, so 9,993 rows stayed. The check is still needed in case new data has repeats.

Outliers (Part 5)

```
columns = ["Sales", "Quantity", "Profit"]

for col in columns:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    # count how many values fall outside the limits before capping them
    n_outliers = ((df[col] < lower) | (df[col] > upper)).sum()
    print(f"{col}: lower = {lower:.2f}, upper = {upper:.2f}, values capped = {n_outliers:,} ({n_outliers / len(df):.1%})")

    df[col] = df[col].clip(lower, upper)

print("\nOutliers handled.")
```

```
Sales: lower = -271.71, upper = 498.93, values capped = 1,167 (11.7%)
Quantity: lower = -2.50, upper = 9.50, values capped = 170 (1.7%)
Profit: lower = -39.72, upper = 70.81, values capped = 1,881 (18.8%)

Outliers handled.
```

Capping Sales, Quantity and Profit at 1.5 × IQR

Column	Lower limit	Upper limit	Values capped
Sales	−271.71	498.93	1,167 (11.7%)
Quantity	−2.50	9.50	170 (1.7%)
Profit	−39.72	70.81	1,881 (18.8%)

Why: extreme values can distort scaling and models, so values outside the 1.5 × IQR limits are capped rather than deleted, which keeps all rows. Discount was left uncapped because its 0–0.8 values are treated as valid observations.

Profit is heavily tailed, and 18.8% of its values were capped, which compresses the largest gains and losses. This is a trade-off: fewer extreme values, but less information about their original magnitude.

Data Types (Part 6)

Dates are converted to datetime for date calculations. Postal Code is an identifier, so it is stored as text rather than used as a numerical measurement.

```python
df['Order Date'] = pd.to_datetime(df['Order Date'], errors = 'coerce')
df['Ship Date'] = pd.to_datetime(df['Ship Date'], errors = 'coerce')
df["Postal Code"] = df["Postal Code"].astype("string").str.zfill(5)

print(df.dtypes)
```

```
Row ID                 int64
Order ID                 str
Order Date    datetime64[us]
Ship Date     datetime64[us]
Ship Mode                str
Customer ID              str
Customer Name            str
Segment                  str
Country                  str
City                     str
State                    str
Postal Code           string
Region                   str
Product ID               str
Category                 str
Sub-Category             str
Product Name             str
Sales                float64
Quantity             float64
Discount             float64
Profit               float64
dtype: object
```

Converting dates to datetime and Postal Code to text

New Features (Part 7)

```python
df["Order Year"] = df["Order Date"].dt.year
df["Order Month"] = df["Order Date"].dt.month
df["Order Quarter"] = df["Order Date"].dt.quarter
df["Shipping Days"] = (df["Ship Date"] - df["Order Date"]).dt.days

df["Profit Status"] = df["Profit"].apply(
    lambda x: "Profit" if x > 0 else "Loss" if x < 0 else "Break-even"
)

df["Profit Margin"] = df_raw["Profit"] / df_raw["Sales"]
df["Unit Price"] = df_raw["Sales"] / df_raw["Quantity"]

df["Order Weekday"] = df["Order Date"].dt.dayofweek


print(df[[
    "Order Year",
    "Order Month",
    "Order Quarter",
    "Shipping Days",
    "Profit Status",
    "Profit Margin",
    "Unit Price",
    "Order Weekday"
]].head())
```

```
   Order Year  Order Month  Order Quarter  Shipping Days Profit Status  \
0        2016           11              4              3        Profit
1        2016           11              4              3        Profit
2        2016            6              2              4        Profit
3        2015           10              4              7          Loss
4        2015           10              4              7        Profit

   Profit Margin  Unit Price  Order Weekday
0         0.1600    130.9800              1
1         0.3000    243.9800              1
2         0.4700      7.3100              6
3        -0.4000    191.5155              6
4         0.1125     11.1840              6
```

Creating eight new features (first rows shown)

Feature	What it shows
Order Year, Order Month, Order Quarter	Yearly, monthly and seasonal sales patterns
Order Weekday	Day of the order (0 = Monday, 6 = Sunday)
Shipping Days	Delivery speed, average 3.96 days
Profit Status	Profit, Loss or Break-even: 80.6% profit, 18.7% loss, 0.7% break-even
Profit Margin	Profit divided by Sales
Unit Price	Sales divided by Quantity

Why: each feature adds useful information such as timing, shipping speed and profitability. Profit Margin and Unit Price use the original (uncapped) values from `df_raw`, because ratios of capped values would be wrong. The date features and Profit Status are not affected by capping.

Encoding Categorical Variables (Part 8)

```
# ------Convert text categories into numerical columns-----
df = pd.get_dummies(df, columns = ["Category", "Segment", "Region", "Ship Mode"], dtype = int)
```

One-hot encoding four categorical columns

Why: models need numbers, so each category becomes its own 0/1 column. Category (3), Segment (3), Region (4) and Ship Mode (4) became 14 columns.

Scaling Numerical Features (Part 9)

```
# -----Use StandardScaler to standardized the selected numerical columns-----
scaler = StandardScaler()
columns = ["Sales", "Quantity", "Discount", "Profit"]
df[columns] = scaler.fit_transform(df[columns])

print(df[columns].head())
```

✓ 0.0s

	Sales	Quantity	Discount	Profit
0	0.720937	-0.834027	-0.756554	0.876514
1	2.124802	-0.358399	-0.756554	1.856713
2	-0.744362	-0.834027	-0.756554	-0.312014
3	2.124802	0.592857	1.423182	-1.892206
4	-0.698461	-0.834027	0.212218	-0.459723

Standardizing to mean 0 and standard deviation 1 (first rows shown)

Why: puts the features on the same scale so large values like Sales do not dominate.

Note: scaling here is fitted on all rows, so cleaned_superstore.csv is meant for analysis. For model training, use the pipeline in Part 11, which fits the scaler on the training split only to avoid data leakage.

Descriptive Statistics After Preprocessing (Part 10)

```
df[["Sales","Quantity","Discount","Profit"]].describe().T
```

Descriptive statistics after preprocessing

Column	Mean	Std	Median	Min	Max
Sales	0.00	1.00	−0.51	−0.83	2.12
Quantity	0.00	1.00	−0.36	−1.31	2.73
Discount	0.00	1.00	0.21	−0.76	3.12
Profit	0.00	1.00	−0.25	−1.89	1.86

The values are now standardized (mean ≈ 0, standard deviation ≈ 1). Sales, Quantity and Profit were capped before scaling, so the original Sales maximum (about 22,638) is no longer present: the capped maximum scales to 2.12. Likewise the original minimum Profit (about −6,600) was capped, and the capped minimum scales to −1.89.

Discount was not capped, so its higher values still stand out in the box plot (maximum 3.12).

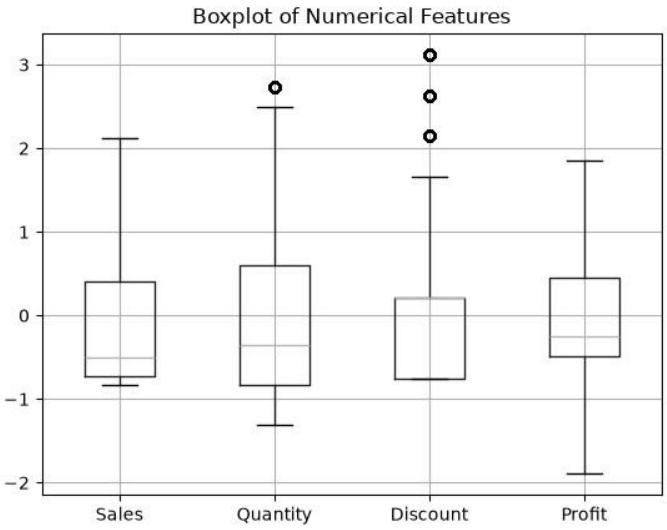


Figure 1: Box plot of the four numerical features after preprocessing

Scikit-learn Preprocessing Pipeline – Bonus (Part 11)

A reusable pipeline automates imputing, encoding and scaling. Numeric features receive median imputation and StandardScaler; categorical features receive most-frequent-value imputation and OneHotEncoder.

```
data = pd.read_csv("superstore.csv", encoding = "latin-1")

# -----Select a few useful features-----
X = data[["Sales", "Quantity", "Discount", "Profit", "Category", "Region"]]

# -----Separate numerical and categorical columns-----
num_cols = ["Sales", "Quantity", "Discount", "Profit"]
cat_cols = ["Category", "Region"]

# -----Numerical preprocessing-----
num_pipe = Pipeline([
    ("fill", SimpleImputer(strategy = "median")),
    ("scale", StandardScaler())
])

# -----Categorical preprocessing-----
cat_pipe = Pipeline([
    ("fill", SimpleImputer(strategy = "most_frequent")),
    ("encode", OneHotEncoder(handle_unknown = "ignore"))
])
```

```python
# -----Combine both pipelines-----
preprocessor = ColumnTransformer([
    ("numbers", num_pipe, num_cols),
    ("categories", cat_pipe, cat_cols)
])

# -----Split data before fitting the preprocessing pipeline-----
X_train, X_test = train_test_split(
    X, test_size = 0.2, random_state = 42
)

# -----Fit on training data and transform both sets-----
X_train = preprocessor.fit_transform(X_train)
X_test = preprocessor.transform(X_test)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

Reusable Pipeline and ColumnTransformer

```
Training data: (7995, 11)
Testing data: (1999, 11)
```

Shapes of the processed training and testing data

The data is split 80/20 first and the pipeline is fitted on the training set only, which helps prevent data leakage. The result is 7,995 training rows and 1,999 test rows with 11 processed features: 4 numeric features, 3 one-hot Category columns and 4 one-hot Region columns.

This pipeline is a separate modelling example, not the same transformation as the cleaned CSV export. It uses only Category and Region as categorical inputs, while the full workflow encodes Category, Segment, Region and Ship Mode, so the two outputs are not expected to have the same number of columns. It is also fitted on the raw file (9,994 rows, before duplicate removal), so its row counts differ by one from the cleaned CSV (9,993 rows).

Final Cleaned Dataset

```python
df = df.drop(columns = ["Row ID", "Country"])
df.to_csv("cleaned_superstore.csv", index = False)

print("Cleaned dataset saved successfully!")
```

```
Cleaned dataset saved successfully!
```

Saving the final dataset

The final dataset, cleaned_superstore.csv, has 9,993 rows and 37 columns. Row ID (a simple counter) and Country (one constant value) were dropped.

Stage	Columns
Raw dataset	21
Row ID and Country dropped	19
8 new features added	27
4 columns one-hot encoded into 14	37

Challenges Faced

- The dataset had no missing values and only one duplicate (Row IDs 3406 and 3407, identical except for the ID), so the fill code protects future data while the duplicate check removed one row.
- Sales and Profit are heavily skewed, so a rule had to be chosen that removes the extreme-value problem without losing any rows beyond the one duplicate. Capping at 1.5 × IQR does this.
- Quantity became a decimal column after capping, because the IQR limits are decimals. It is still a count and was standardized like the others.
- Features had to be created before scaling so that Profit Margin and Unit Price stay readable.
- Capping changes Sales and Profit separately, so ratio features built from capped values would not match the true ratios. Profit Margin and Unit Price are therefore calculated from the original columns.
- Postal Code looks numeric but is an identifier, so it was stored as text instead of being scaled.

Conclusion

The raw Superstore data is now clean, consistent and ready for analysis or modelling. Outliers were capped without losing any rows, dates and IDs have correct types, eight new features add timing, delivery and profitability information, categories are encoded and the four main numeric columns are standardized.

A reusable Scikit-learn pipeline repeats the imputing, encoding and scaling steps on new data. The final file has 9,993 rows and 37 columns.

GitHub Repository Link

https://github.com/duasajid65-cloud/DevixoSolutions-Internship